

CampusOS AI

Your material. Your language. Offline.

A private study assistant that turns a student's own course PDFs into a voice-driven learning loop — cited explanations in their own language, then quizzes that reveal what they actually don't know.

Snapdragon® AI Lab Build & Present Challenge · Solution Submission Round

Built for Snapdragon-powered HP PCs · github.com/abhayjeet5465/campusOs

Four problems at once. Tools solve two.

1

The material is theirs

The syllabus, the professor's slides, last year's paper. A general chatbot has never seen any of it.

2

They cannot verify

A fluent wrong answer looks exactly like a right one — to the person still learning the topic.

3

Wrong language register

Notes are in English. Understanding happens in Hinglish. They translate in their heads, under time pressure.

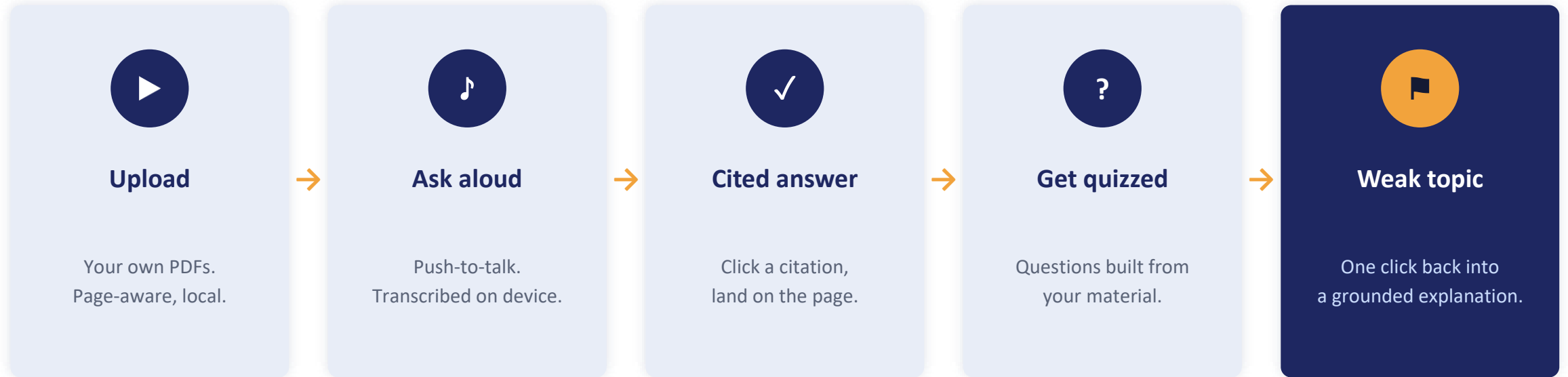
4

They don't know their gaps

Re-reading feels like studying. It is the worst way to find what you don't know.

And a fifth: hostel Wi-Fi fails exactly during the 1 a.m. sessions when the help is needed most.

The loop is the product — not the chat



Most tools stop at the answer.

Re-reading is the least effective way to find a gap. CampusOS closes the loop: it quizzes you, names the topic you actually failed, and links that topic straight back into an explanation drawn from your own pages.

It refuses. And it cannot fake a citation.



The refusal gate

If retrieval scores below threshold, the answer is refused before the language model is ever called — so the model cannot talk its way past it. The student is shown what was found nearby instead.



Citation validation

Every [doc, p.N] marker is checked against the chunks actually supplied to the model. A page reference it invented is stripped before the student ever sees it.

The system is built so that it cannot cite a page it was not shown.

Enforced in code and covered by tests — not requested in a prompt. This matters because the students who most need this tool are exactly the ones least able to catch a confident hallucination.

One codebase. Two runtime configurations.

Workload	Interface	Dev host (x64)	Snapdragon HP PC
Language model	OpenAI-compatible HTTP	llama.cpp (CPU)	GenieX / llama.cpp ARM64
Embeddings	ONNX Runtime session	CPUExecutionProvider	QNNExecutionProvider
Speech-to-text	ONNX Runtime session	faster-whisper (CPU)	AI Hub Whisper via QNN
Retrieval	NumPy cosine	CPU	CPU — sub-millisecond



Porting is configuration, not engineering.

The language model runs out-of-process behind an OpenAI-compatible endpoint, so swapping in GenieX is an environment variable. ONNX providers are chosen at startup by platform detection — QNN first, CPU immediately after in the same list, so an unsupported graph degrades instead of crashing. No application code changes.

What we did — and what we won't claim



Done, in the shipped code

- QNN execution-provider selection implemented, not described — active provider reported at `/api/health`
- Same GGUF weights run under `llama.cpp` on x64 and GenieX on Snapdragon, with no conversion step
- Benchmark harness runs unmodified on ARM64, so both columns are directly comparable
- Full ARM64 setup guide ships in the repo — `onnxruntime-qnn` swap, `QnnHtp.dll` path, how to verify QNN loaded



Not measured — stated plainly

No NPU execution has been measured. This development machine has no NPU, and per ONNX Runtime's documentation the win-x64 build supports QNN quantization and AOT compilation only — not inference.

Every NPU statement here is a documented deployment path with a citation. The Snapdragon column in our results table is deliberately empty and labelled "requires hardware".

We would rather show an honest gap a judge can verify than a projected number that would not survive scrutiny.

Every claim is a command a judge can run

0

outbound connections

during a full pipeline run, asserted by test

11.9

pages / second

ingest throughput, measured

~8 ms

retrieval latency

brute-force cosine, no index needed

`tests\test_egress.py`

Blocks every non-loopback socket, runs the full pipeline anyway

`tests\test_pipeline.py`

Ingest, retrieval, the refusal gate, and hard deletion

`benchmarks\run.py`

Regenerates the results table, including the active execution provider

`eval\run_eval.py`

Groundedness and out-of-corpus refusal against a fixed evaluation set

Nothing leaves the laptop



No accounts, no keys, no cloud

There is no authentication code anywhere in the repository, because there is nothing to authenticate to.



No telemetry of any kind

No analytics, no crash reporting. The only network event in the product's life is the first-run model download.



Audio is never stored

Transcribed, then the buffer is discarded. The transcript is shown for confirmation before it is used.



Delete means delete

SQLite rows removed and the vector array rebuilt without them — asserted by test, not assumed.

Runs within an 8 GB RAM floor · one command to start · fully keyboard-operable — voice is never the only path

**A tutor that has read your syllabus,
speaks your language, shows its sources,
and works with the Wi-Fi off.**

Because the model runs on the machine in front of them, not in someone else's datacentre.

Grounded or silent

Cite, always

Local by default

Teach the gap

github.com/abhayjeet5465/campusOs

Built with Llama. Models are downloaded from their original publishers and are not redistributed.